

Enrichissement de Leads B2B par Lots

n8n — IF, Switch, Loop Over Items, Continue On Fail, Error Trigger

Durée estimée

environ 3 heures

Fichier de départ

`sujet_workflow.json` (à importer dans n8n)

Prérequis

un accès n8n; aucun compte externe, aucun credential

Rendu

envoi automatique à la moulinette depuis le workflow

Table des matières

1	Contexte	2
2	Spécifications	2
3	Le fichier de départ	2
4	Le jeu de données	3
5	Instructions pas à pas	3
	Étape 1 — Inspecter la base (<i>rien à modifier</i>)	4
	Étape 2 — Éclater le tableau — nœud Split Out	4
	Étape 3 — Filtrer les leads à traiter — nœud If	4
	Étape 4 — Router par secteur — nœud Switch	4
	Étape 5 — Traiter les Tech par lots — nœud Loop Over Items	4
	Étape 6 — Brancher l’enrichissement (<i>nœud fourni</i>)	5
	Étape 7 — Marquer le résultat et reboucler — nœud Marquer le resultat	5
	Étape 8 — Les secteurs non enrichis — deux nœuds Edit Fields	6
	Étape 9 — Réconcilier — nœud Merge	6
	Étape 10 — Brancher la sortie (<i>nœuds fournis</i>)	6
	Étape 11 — Capturer les crashes globaux — Error Trigger	6
6	Résultats attendus	7

1 Contexte

Vous êtes responsable des opérations (Ops) d'une agence B2B. Votre équipe commerciale a collecté **20 leads bruts**. Votre mission consiste à compléter un flux n8n qui écarte les leads qui ne sont pas à traiter, les trie par secteur, enrichit **uniquement les profils Tech** via une API externe, réconcilie l'ensemble, puis produit un fichier CSV et transmet le résultat à la moulinette de correction.

Deux contraintes structurent tout l'exercice. D'abord, l'API d'enrichissement est **limitée en débit** : vous devez l'appeler **par lots de 5**. Ensuite, elle est **incomplète** : elle ne connaît pas tous vos leads. Votre flux doit donc encaisser ces échecs **sans s'arrêter** — c'est tout l'objet du réglage *Continue On Fail*.

2 Spécifications

Paramètre	Consigne
Durée estimée	environ 3 heures
Fichier de départ	<code>sujet_workflow.json</code>
Nœuds à ajouter	Split Out, If, Switch, Loop Over Items, Edit Fields (×3), Merge, No Operation
Taille des lots	5 items par itération
Gestion d'erreurs locale	Continue On Fail sur le nœud HTTP d'enrichissement (<i>déjà configuré</i>)
Gestion d'erreurs globale	un second workflow déclenché par Error Trigger
API d'enrichissement	<code>https://jsonplaceholder.typicode.com/users/{id}</code>
Rendu	POST vers <code>https://n8n.maikagency.dev/webhook/moulinette</code>

3 Le fichier de départ

Importez `sujet_workflow.json` dans n8n via *Workflows* → *Import from File*.

Ce qui est déjà fourni — n'y touchez pas

Nœud	Rôle
Execute workflow Manually	Déclencheur manuel, déjà branché sur Database
Database	Contient la variable <code>lead_non_enrichi</code> : vos 20 leads
HTTP Request - Enrichissement	Appel d'enrichissement, déjà configuré
Exporter vers .csv	Conversion du résultat final en fichier CSV
Aggregate	Regroupe les items en une liste, déjà branché sur la moulinette
Send to moulinette	Envoi du rendu, déjà configuré

Ce que vous devez faire

Ajouter des nœuds et créer autant de connexions que vous voulez. Trois nœuds fournis attendent d'être reliés : la sortie de Database n'est branchée nulle part, le nœud HTTP Request - Enrichissement n'a ni entrée ni sortie, et Exporter vers .csv comme Aggregate n'ont pas d'entrée.

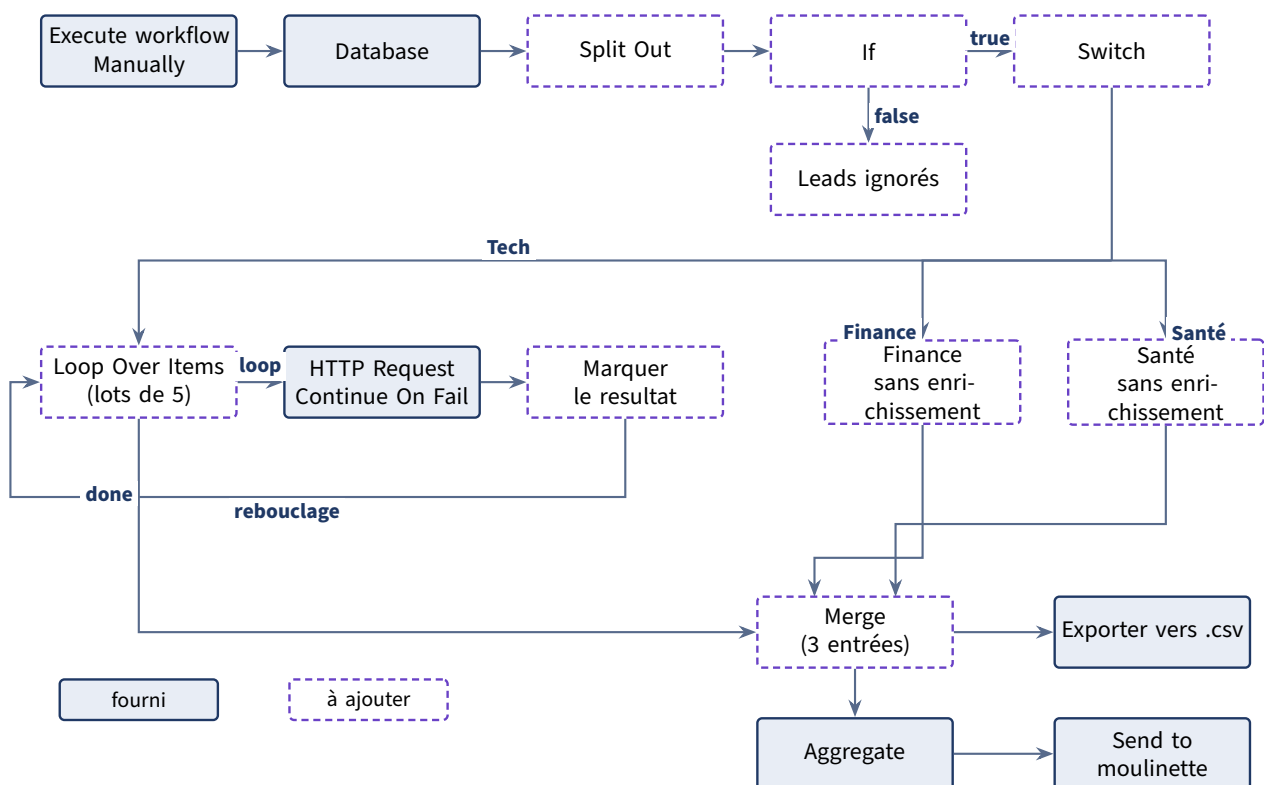


FIGURE 1 – Le flux complet. Cadres pleins : nœuds fournis. Cadres pointillés : nœuds à créer. Le corps de boucle est linéaire — un seul nœud reboucle vers Loop Over Items.

4 Le jeu de données

Les 20 leads sont déjà dans le nœud `Database`. Ils y sont stockés dans une variable de type `Array` nommée `lead_non_enrichi`, et chaque lead porte six champs.

Colonne	Rôle
<code>id</code>	identifiant du lead et clé de recherche chez le fournisseur d'enrichissement
<code>nom, email</code>	identité du contact
<code>statut</code>	nouveau (à traiter) · contacté · invalide
<code>secteur</code>	Tech · Finance · Santé
<code>enrichi</code>	FAUX au départ; deviendra VRAI, ERREUR, ou restera FAUX

⚠ À comprendre avant de commencer

L'API <https://jsonplaceholder.typicode.com/users/{id}> ne connaît que les **id 1 à 10**. Au-delà, elle répond **HTTP 404**.

Dans votre base, les `id` sont ceux du CRM : ils ne sont **ni contigus ni triés** (13, 5, 21, 1, 17, 9, ...), et les leads réellement connus du fournisseur sont **dispersés** dans la liste. **Vous ne pouvez pas deviner le résultat en lisant la base** : il faut exécuter le flux.

5 Instructions pas à pas

1 Inspecter la base (rien à modifier)

Ouvrez **Database** et repérez la variable `lead_non_enrichi`. Exécutez le workflow une première fois : le nœud sort **un seul item** contenant un tableau de 20 leads.

2 Éclater le tableau — nœud Split Out

Ajoutez un nœud **Split Out** en gardant le nom par défaut `Split Out`, et renseignez **Field to Split Out** avec `lead_non_enrichi`.

🔗 **Branchez :** Database → Split Out

✅ **Sortie attendue :** 20 items, un par lead.

Sans cette étape, toute la suite travaillerait sur **un seul** item contenant un tableau.

3 Filtrer les leads à traiter — nœud If

Ajoutez un nœud **If** (nom par défaut `If`) avec la condition `{{ $json.statut }}` — *String* — **is equal to** — `nouveau`. Ajoutez également un nœud **No Operation** renommé `Leads ignorés`.

🔗 **Branchez :** Split Out → If; puis If (**false**) → Leads ignorés

✅ **Sortie attendue :** 15 en **true**, 5 en **false** (3 contacté + 2 invalide).

4 Router par secteur — nœud Switch

Ajoutez un nœud **Switch** (nom par défaut `Switch`). Créez trois routes sur `{{ $json.secteur }}` (*is equal to*) et **renommez chaque sortie**. Dans **Options**, laissez **Fallback Output** sur `None`.

Route	Valeur	Nom de sortie
0	Tech	Tech
1	Finance	Finance
2	Santé	Santé

🔗 **Branchez :** If (**true**) → Switch

Piège

Santé s'écrit **avec l'accent**, exactement comme dans la base. Sans lui, la route reste vide.

✅ **Sortie attendue :** Tech 11 · Finance 2 · Santé 2 (total 15).

5 Traiter les Tech par lots — nœud Loop Over Items

Ajoutez un nœud **Loop Over Items** et **gardez impérativement le nom par défaut** `Loop Over Items` : il est référencé par une expression à l'étape 7. Réglez **Batch Size** sur **5**.

🔗 **Branchez :** Switch (route **Tech**) → Loop Over Items

Ce nœud possède **deux sorties** :

Sortie	Nom	Quand ?
0	done	une seule fois , quand tous les lots sont traités
1	loop	une fois par lot

✔ **Attendu** : 11 leads Tech en lots de 5.

6 Brancher l'enrichissement (nœud fourni)

Le nœud `HTTP Request - Enrichissement (Continue On Fail)` est **déjà configuré**. Ouvrez-le pour comprendre ce qui a été fait.

Réglage	Valeur
URL	<code>https://jsonplaceholder.typicode.com/users/{{ \$json.id }}</code>
Options → Timeout	<code>10000</code>
Settings → On Error	Continue (using regular output)
Settings → Always Output Data	activé

🔗 **Branchez :** `Loop Over Items (sortie loop)` → `HTTP Request - Enrichissement`

Sans *Continue On Fail*, le premier `404` **arrêterait tout le workflow**. Avec, le lead en échec ressort sur la **sortie normale** sous la forme `{ "error": ... }`, et le flux continue.

✔ **Attendu** : à vous de vérifier.

7 Marquer le résultat et reboucler — nœud Marquer le resultat

Ajoutez un nœud **Edit Fields (Set)** renommé `Marquer le resultat`, puis reconstituez les six colonnes du lead.

Champ	Type	Valeur
id	Number	<code>{{ \$('Loop Over Items').item.json.id }}</code>
nom	String	<code>{{ \$('Loop Over Items').item.json.nom }}</code>
email	String	<code>{{ \$('Loop Over Items').item.json.email }}</code>
statut	String	<code>{{ \$('Loop Over Items').item.json.statut }}</code>
secteur	String	<code>{{ \$('Loop Over Items').item.json.secteur }}</code>
enrichi	String	<code>{{ \$json.error !== undefined ? 'ERREUR' : 'VRAI' }}</code>

🔗 **Branchez :** `HTTP Request - Enrichissement` → `Marquer le resultat`, puis **rebouclez** `Marquer le resultat` → `Loop Over Items`

Pourquoi ce nœud ?

La réponse HTTP **remplace** l'item : après l'appel, `$json` contient l'utilisateur renvoyé par l'API, plus votre lead. On récupère donc les champs d'origine depuis le nœud `Loop Over Items`, et on en déduit `enrichi`.

8 Les secteurs non enrichis — deux nœuds `Edit Fields`

Les routes `Finance` et `Santé` ne passent pas par l'API. Il faut néanmoins leur donner **la même forme** que les leads `Tech` pour pouvoir les fusionner ensuite.

Ajoutez deux nœuds **Edit Fields (Set)** renommés `Finance - sans enrichissement` et `Santé - sans enrichissement`. Chacun recopie `id`, `nom`, `email`, `statut` et `secteur` via `{{ $json.<champ> }}`, avec les mêmes types qu'à l'étape 7, et fixe `enrichi` à `FAUX`.

🔗 **Branchez :** Switch (**route Finance**) → `Finance - sans enrichissement`; Switch (**route Santé**) → `Santé - sans enrichissement`

9 Réconcilier — nœud `Merge`

Ajoutez un nœud **Merge** (nom par défaut `Merge`), en mode **Append** avec **Number of Inputs** à **3**.

Entrée	Source
0	Loop Over Items — sortie <code>done</code>
1	<code>Finance - sans enrichissement</code>
2	<code>Santé - sans enrichissement</code>

✅ **Sortie attendue : 15 lignes** (11 `Tech` + 2 `Finance` + 2 `Santé`).

10 Brancher la sortie (nœuds fournis)

Branchez la sortie de `Merge` **vers les deux nœuds** : `Exporter vers .csv` et `Aggregate`.

`Exporter vers .csv` (*Convert to File*, opération **Convert to CSV**) produit un fichier binaire : téléchargez-le et vérifiez vos **15 lignes**. `Aggregate` (*All Item Data (Into a Single List)*) regroupe les 15 items en une seule liste, puis `Send to moulinette` la poste automatiquement — ces deux nœuds sont **déjà branchés entre eux et configurés** : méthode `POST`, URL `https://n8n.maikagency.dev/webhook/moulinette`, et un unique paramètre de corps `data` valant `{{ $json.data }}`.

🔗 **Branchez :** `Merge` → `Exporter vers .csv` et `Merge` → `Aggregate`

11 Capturer les crashes globaux — `Error Trigger`

Créez un **second workflow**, par exemple `Error Handler - Alerte Globale`. Son point de départ est le nœud **Error Trigger**. Ajoutez ensuite un **Edit Fields (Set)** qui extrait les informations de l'incident, puis un nœud d'alerte (`Slack`, `Discord`, `Gmail`, ou un simple `HTTP Request`). Retournez enfin dans le workflow principal, section **Settings** → **Error Workflow**, et sélectionnez ce nouveau workflow.

Champ	Expression
<code>workflow_id</code>	<code>{{ \$json.workflow.id }}</code>
<code>workflow_nom</code>	<code>{{ \$json.workflow.name }}</code>
<code>execution_id</code>	<code>{{ \$json.execution.id }}</code>
<code>noeud_en_erreur</code>	<code>{{ \$json.execution.lastNodeExecuted }}</code>
<code>message_erreur</code>	<code>{{ \$json.execution.error.message }}</code>

Test

Désactivez *Continue On Fail* sur le nœud `HTTP` et relancez. Le premier `404` fait planter l'exécution, et le workflow d'erreur doit démarrer tout seul. Pensez à **réactiver** *Continue On Fail* ensuite.

6 Résultats attendus

Étape	Résultat
Split Out	20 items
If	15 en <code>true</code> · 5 en <code>false</code>
Switch	Tech 11 · Finance 2 · Santé 2
Loop Over Items	3 itérations (5 + 5 + 1) — le nœud s'exécute 4 fois (3 lots + <code>done</code>)
HTTP Request	à vous de vérifier la véracité des résultats à partir du retour de la moulinette
Merge	15 lignes , avec les 6 colonnes <code>id</code> , <code>nom</code> , <code>email</code> , <code>statut</code> , <code>secteur</code> , <code>enrichi</code>
Aggregate	une liste <code>data</code> de 15 éléments

Le détail ci-dessous n'est à consulter **qu'après** votre première exécution.

Valeur	id concernés
VRAI	2, 4, 5, 7, 9, 10
ERREUR	13, 24, 36, 45, 57
FAUX	21, 33, 41, 48
Ignorés par le If	1, 6, 17, 29, 52